

```

import numpy as np

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

# -----

# STEP 1: Take 20 numbers dynamically from user
# -----

user_input = input("Enter 20 numbers separated by spaces: ")

numbers = list(map(float, user_input.split()))

# Check whether exactly 20 numbers are entered
if len(numbers) != 20:
    print("ERROR: Please enter exactly 20 numbers.")
else:

    # -----

    # STEP 2: Create parity feature
    # 0 = Even
    # 1 = Odd
    # -----

    parity = np.array([int(number) % 2 for number in numbers])

    # Convert into 2D format for K-Means
    X = parity.reshape(-1, 1)

    # -----

```

```

# STEP 3: Apply K-Means with 2 clusters
# -----

kmeans = KMeans(
    n_clusters=2,
    random_state=42,
    n_init=10
)

cluster_labels = kmeans.fit_predict(X)

# -----
# STEP 4: Identify which K-Means cluster is Even/Odd
# -----

cluster_centers = kmeans.cluster_centers_.flatten()

even_cluster = np.argmin(cluster_centers)
odd_cluster = np.argmax(cluster_centers)

# -----
# STEP 5: Separate Even and Odd numbers
# -----

even_numbers = [
    numbers[i]
    for i in range(20)
    if cluster_labels[i] == even_cluster
]

```

```

odd_numbers = [
    numbers[i]
    for i in range(20)
    if cluster_labels[i] == odd_cluster
]

# -----
# STEP 6: Display Results
# -----

print("\n=====")
print("    K-MEANS RESULT")
print("=====")

print("\nEven Number Cluster:")
print(even_numbers)

print("\nOdd Number Cluster:")
print(odd_numbers)

print("\nCluster Centers:")
print("Even Cluster Center:", cluster_centers[even_cluster])
print("Odd Cluster Center :", cluster_centers[odd_cluster])

# -----
# STEP 7: Visualization
# -----

```

```
plt.figure(figsize=(10, 6))
```

```
# Plot Even numbers
```

```
even_positions = [  
    i + 1  
    for i in range(20)  
    if cluster_labels[i] == even_cluster  
]
```

```
plt.scatter(  
    even_positions,  
    even_numbers,  
    marker='o',  
    s=100,  
    label='Even Numbers'  
)
```

```
# Plot Odd numbers
```

```
odd_positions = [  
    i + 1  
    for i in range(20)  
    if cluster_labels[i] == odd_cluster  
]
```

```
plt.scatter(  
    odd_positions,  
    odd_numbers,  
    marker='s',  
    s=100,
```

```
label='Odd Numbers'  
)  
  
# Labels  
plt.xlabel("Input Number Position")  
plt.ylabel("Number")  
plt.title("K-Means Clustering of Even and Odd Numbers")  
  
plt.legend()  
plt.grid(True, linestyle='--', alpha=0.5)  
  
plt.show()
```